Highlights

Evidence contracts that cannot be violated: enforcing quality obligations by accounting rather than inference in retrieval-augmented knowledge services

Kang Yao, Shengjiang Zhang, Ronghao Pei, Weiwei Fu, Jinjiang Cui

- A quality contract can be enforced exactly, without any statistical assumption
- Declining is an optimizer action, making contracts below the risk floor feasible
- Removing validity from the estimate lets the optimizer steer far more aggressively
- A larger plan space helps or hurts a certificate unpredictably; the ledger is immune
- Cost, declines and audit labels are metered end to end on four benchmarks

Evidence contracts that cannot be violated: enforcing quality obligations by accounting rather than inference in retrieval-augmented knowledge services

Kang Yao[a,b,1], Shengjiang Zhang[a,b,1], Ronghao Pei[c], Weiwei Fu[b,*] and Jinjiang Cui[b,*]

[a]School of Biomedical Engineering (Suzhou), Division of Life Sciences and Medicine, University of Science and Technology of China, 166 Ren'ai Road, Suzhou Industrial Park, Suzhou, 215123, China

[b]Suzhou Institute of Biomedical Engineering and Technology, Chinese Academy of Sciences, No. 88 Keling Road, Suzhou New District, Suzhou, 215163, China

[c]Hikvision Research Institute, Hangzhou, China

ARTICLE INFO

Keywords:
Retrieval-augmented generation
Knowledge service reliability
Selective prediction
Online resource allocation
Distribution-free inference
Query optimization

ABSTRACT

A deployed retrieval-augmented generation (RAG) service is held to a *rate*: no more than a stated fraction of its answers may be unsupported, and the bill should be as small as possible subject to that. The literature discharges such obligations by inference – estimate each candidate pipeline's risk on a calibration sample, correct for multiplicity, deploy one whose estimate clears the level minus a confidence width. We identify three structural costs of that choice: the width is paid on every query forever, it grows with the number of candidates, so that enlarging the plan space changes the certified cost by an amount whose sign is task-dependent and cannot be known before the space is built, and any level below the best pipeline's risk is infeasible by construction – which, on our benchmarks, excludes every contract an operator would write. We show the obligation can instead be discharged by accounting. Admitting a decline action, whose contract loss is zero, and gating execution on the ledger $B_t = \alpha t - V_t$ of unspent allowance yields $V_t \le \alpha t$ at every t, on every path, under any query order, independently of how many plans were considered – no probability, no exchangeability, no horizon. Statistics then serves cost alone: we give a linear program over plans plus declining, prove $O(\sqrt{P \log T / T})$ cost regret, and price partial auditing exactly. On fully metered executions of HybridQA, CRAG, ASQA and QAMPARI over a plan space spanning three model developers and three open-weight models, the executor runs at 1.27–1.75× an offline optimum that knows every plan's risk, against 2.89–5.85× for one-shot certification given the same action space. Sweeping the contract over 15 levels per task, it honours every level on every stream while spending 92–99% of the allowance, where the certifier breaches 545 of 1 200 streams; handed the decline action the certifier stops breaching and its bill does not move (5.49× against 5.47× the optimum), because a confidence width is paid whether it is spent on a plan or on refusing. Under a mid-stream distribution shift the certificate is breached on up to 100% of streams while the ledger is breached on none, and tightening the certifier's error budget by 200× moves its breach count in one of twelve cells – its failure is a calibration sample that misrepresents the stream, which no confidence correction repairs. Enforcement costs 14–49 microseconds per query against 0.8–2.8 seconds of retrieval and generation.

1. Introduction

A knowledge service built on retrieval-augmented generation (RAG) is usually asked for something its designers cannot promise. A compliance office does not ask that a particular answer be correct; it asks that no more than, say, one answer in ten be wrong, and that the bill be as small as possible subject to that. The obligation is a *rate* over a stream of queries, it is checked on the service log, and it is the kind of thing that goes into a service-level agreement.

The literature discharges this obligation by inference. Take a calibration sample, estimate the risk of each candidate pipeline, correct for having examined many candidates, and deploy one whose estimate clears α minus a confidence width. Distribution-free risk control [1, 2] makes this rigorous, and conformal methods for RAG [3, 4] apply it to retrieval pipelines. The resulting statement is: the deployed policy's risk is at most α with probability $1 - \delta$, provided deployment traffic is exchangeable with the calibration sample.

Three costs come with that statement, and this paper is about the fact that all three are structural rather than incidental.

The confidence width is paid forever. A calibration-time decision cannot be revised, so it must be right the first time, so it must be conservative by the full estimation error. That slack is $\Theta(\sqrt{\log(K/\delta)/n_{\text{cal}}})$, it is fixed the moment calibration ends, and it is charged on every query for the life of the deployment. On our data it is 0.085 at α levels of 0.15–0.34, which is not a correction but a redefinition of the contract.

Enlarging the plan space helps or hurts, and nobody can say which in advance. The width grows with the number of candidates, while the plans themselves get cheaper.

*Weiwei Fu and Jinjiang Cui are co-corresponding authors.

yaokang@mail.ustc.edu.cn (K. Yao); zhangshengjiang@mail.ustc.edu.cn (S. Zhang); peironghao@hikvision.com (R. Pei); fuww@sibet.ac.cn (W. Fu); cuijj@sibet.ac.cn (J. Cui)

ORCID(s): 0009-0007-4747-4018 (K. Yao); 0009-0003-6744-743X (S. Zhang); 0000-0001-7192-3118 (W. Fu); 0009-0002-5159-9570 (J. Cui)

[1]Kang Yao and Shengjiang Zhang contributed equally to this work.

We built plan grids by unbundling retrieval configuration from generator choice – 36 plans on one benchmark, 24 on another, spanning three model developers and three locally served open-weight models – and certified each with the same procedure used on the conventional 4-rung ladder it contains. In the offline problem the larger space wins every time. Under certification the sign flips with the task: on one benchmark the certified policy came out 19% *more* expensive than on the ladder, on the other 31% cheaper, and each space has a contract level at which it certifies nothing while the other certifies a policy. A query optimizer whose search space cannot be enlarged with a predictable effect is not a query optimizer.

Strict contracts are infeasible by construction. The attainable risk of a system that must answer every query is bounded below by the risk of its best pipeline. That floor is 0.59 on two of our four benchmarks and 0.27 on a third, so a 10% contract – the one the compliance office actually wrote – is not merely hard to certify, it is unexpressible. No amount of statistical sharpening moves a floor.

The third point is the one that reveals what is wrong. Infeasibility is not a statistical failure; it says the action space is too small. Real services are not obliged to answer every query. They may decline – return "insufficient evidence", route to a human, degrade to a document list. Declining yields no wrong answer, so its contract loss is exactly zero, and it is available at every arrival.

Admitting that one action changes the character of the problem. Track the ledger $B_t = \alpha t - V_t$, the contract allowance earned but unspent after t queries and V_t violations, and gate on it: run a plan only when the ledger can absorb a violation, $B_{t-1} \geq 1 - \alpha$; otherwise decline. Then $B_t \geq 0$ for every t by induction, that is,

$$V_t \leq \alpha t \qquad \text{for every } t,$$

with no probability, no distributional assumption, no dependence on how many plans were considered, and no horizon. The contract is not something the system is confident about; it is something the system cannot violate, in the way that an account with a hard overdraft limit cannot go negative.

That leaves cost as the only open problem, and it is a real one: the optimizer must choose, at each arrival, among plans whose risks it does not know, and declining is expensive. But an error in a risk estimate now costs money and nothing else – it drains the ledger faster, the gate closes more often, the bill goes up, and the contract still holds exactly. This is the division of labour a database query optimizer has always had. Cardinality estimates choose plans and are wrong by orders of magnitude [5]; correctness is the executor's responsibility, not the estimator's. Certified RAG has been putting the guarantee on the estimate, and paying for its error forever. Moving it to the executor is the whole of our proposal, and the rest – strict contracts becoming feasible, plan spaces becoming free to enlarge, drift needing no detector – follows rather than being added.

We instantiate this as ContractRAG, a contract-enforcing executor for hybrid RAG, and evaluate it on fully metered executions of four benchmarks (HybridQA, CRAG, ASQA, QAMPARI) over a plan space spanning three commercial vendors and two locally served open-weight models, with every LLM call priced and every label charged.

Contributions.

- **A deterministic contract guarantee.** The ledger gate yields $V_t \leq \alpha t$ pathwise, for every t, under any query order including adversarial ones, independent of the size of the plan space (Theorem 1). We show empirically that this is not a technicality: under a mid-stream distribution shift, a calibration certificate is breached on 100% of streams while the ledger is breached on none of them.
- **Declining as a first-class action, and what it buys.** Adding $\perp$ to the action space makes every $\alpha > 0$ attainable and forces the minimum decline rate $1 - \alpha / \min_p r_p$ when the contract is below the best plan's risk (Proposition 1). Crucially, declining alone is *not* the source of the gain: handed the same action, the static certifier's cost changes by under 2% and on two tasks gets slightly worse, because its confidence width still forces it to over-decline. Ledger plus randomisation is 1.93–4.24× cheaper than the same certifier with the same action space.
- **Cost as a pure learning problem.** With validity removed from the optimisation, the executor may steer aggressively; we give an action-LP formulation over plans plus declining, prove an $O(\sqrt{P \log T / T})$ regret against the offline optimum (Theorem 2), and measure it on streams of up to 32 000 queries, where it falls to 1.5–7.7% of that optimum.
- **Honest accounting for labels.** A deterministic guarantee needs the loss of every served query. We give the exact price of not having it: worst-case accounting preserves $V_t \leq \alpha t$ and pays in declines, inverse-probability accounting preserves the cost and breaches on 6–59% of streams at a 25% label rate (Proposition 2). Audit cost is included in every number we report.
- **A metered, released artifact.** Every execution behind every number – four benchmarks on a four-rung ladder, plus the 32-plan grid – was run end to end, with token-exact pricing for hosted models and GPU-second pricing for local ones; the execution matrices, the call cache and the code regenerate every table without further LLM calls.

Across the four benchmarks, at each task's strictest non-degenerate contract, the executor operates at 1.27–1.75× the cost of an offline optimum that knows every plan's risk, against 2.89–5.85× for one-shot certification given the same action space. Swept over 15 contract levels per task, it honoured every level on every stream where the certifier

breached 545 of 1 200; over every stream in this paper, spanning every task, contract level, decline price and query order we ran, no ledger run ever exceeded its contract.

2. Related work

Distribution-free risk control Conformal prediction [6, 7] and its risk-controlling extensions – RCPS [2], Learn-then-Test [1], conformal risk control [8] – turn a calibration sample into a finite-sample guarantee on a population risk, and are the statistical foundation of certified RAG. Applications to retrieval pipelines include TRAQ [3], C-RAG [4] and conformal factuality [9]; recent work extends the machinery to hallucination filtering [10], and there is work relaxing exchangeability itself by reweighting the calibration sample [11]. All of this yields Definition 2: a bound on an unobservable population mean, valid with probability $1 - \delta$ under exchangeability or a quantified departure from it. Our contribution is orthogonal to how tight such a bound can be made. We argue that for a rate contract over a served stream the population mean is the wrong target, because the quantity an operator is held to is the realised rate on the log, and that quantity can be controlled exactly.

Anytime-valid inference E-values, test martingales and confidence sequences [12–14] give bounds that hold at all stopping times, and are the natural tool for monitoring a deployed model; e-detectors [15] extend them to changepoint detection, and our own earlier design used a restart-mixture e-process to trigger recertification. These methods weaken the fixed-sample assumption but keep the probabilistic form, and they still require the observations to be independent of the past given the model. The ledger needs neither: its guarantee is an accounting identity, so an anytime bound is not something it achieves at a confidence level but something it satisfies by construction. We use a confidence sequence in this paper only to report diagnostics.

The closest work to ours in setting is conformal selective acting [16], which also deploys per round, also lets the system refuse rather than act, and also asks for a guarantee that survives at every time rather than on average. It runs an e-process per candidate threshold and keeps the threshold whose process has not yet rejected, so its bound has the form $R_T \leq \alpha + O(N_T^{-1/2})$ and holds with probability $1 - \delta$. It inherits the usual consequences: a δ to choose, a Bonferroni grid to pay for, a slack term that vanishes only asymptotically, and a statement about the distribution of paths rather than the path in front of you. Our mechanism removes the inference step rather than sharpening it. The gate is arithmetic on the realised loss, so the bound is $V_t \leq \alpha t$ with no additive slack, no δ, and no dependence on the number of candidates; the price is that the loss must be observed, which is why this paper prices auditing explicitly in Section 6.5 rather than treating labels as free. The two designs are complementary: theirs applies when labels are scarce and a probabilistic guarantee is acceptable, ours when labels exist and the obligation is contractual.

Selective prediction and abstention Rejecting inputs rather than predicting on them is classical [17–19] and has been revisited for language models, where conformal factuality filters remove unsupported claims rather than whole answers [9]. That literature asks a predictor to abstain to raise its accuracy on what remains, and characterises the resulting risk–coverage curve. We use abstention differently: not as a property of one model but as an *action* in an optimizer's space, priced alongside the plans and scheduled by a cost objective. This is what makes the risk–cost frontier continuous (Proposition 1) and what makes contracts below the risk floor attainable. The connection to selective prediction is that our decline rate at a strict contract is exactly the coverage that literature would report; the difference is that we choose it to minimise total cost, rather than sweeping it.

Online resource allocation and bandits with knapsacks Serving a stream under a consumable budget is the subject of bandits with knapsacks [20, 21] and online packing [22], where an optimistic LP over arms subject to budget constraints achieves $O(\sqrt{T})$ regret. Algorithm 1 is recognisably in that family, and Theorem 2 adapts the standard analysis. What is different is the role of the budget. In knapsack problems the budget is a resource whose exhaustion ends the episode, and the guarantee is about regret; here the budget *is* the contract, exhaustion means declining rather than stopping, and the guarantee we care about is the pathwise feasibility that the gate enforces regardless of what the LP does. To our knowledge the observation that a quality contract can be enforced this way – and therefore removed from the statistical problem entirely – has not been made.

Cost-aware LLM and RAG pipelines Cascades and routers reduce spend by sending easy queries to cheap models [23–26]; workload-level optimizers such as Abacus [27] choose configurations under a quality constraint using point estimates; systems work on RAG serving optimises latency and throughput by caching intermediate state [28]. These give no feasibility statement, and our experiments confirm that point-estimate selection violates its constraint on a large fraction of draws. Conversely, the certified line of work above fixes the pipeline and does not optimise cost. This paper's setting – minimise cost subject to a contract that must hold on the realised stream – requires both, and the ledger is what lets the two be designed separately.

Query optimization The separation we propose is standard in databases: cardinality estimates choose plans, are wrong by orders of magnitude [5], and correctness is the executor's responsibility rather than the estimator's; adaptive and robust query processing revises decisions mid-execution [29–31]. Bringing that discipline to RAG requires deciding what plays the role of the executor's correctness invariant, and the answer here is the ledger. The plan-grid experiment of Section 6.6 makes the analogy concrete in the other direction too: under certification-by-inference, enlarging the

search space can degrade the chosen plan, which is a failure mode no database optimizer would tolerate.

3. Preliminaries and problem formulation

We model hybrid RAG pipelines as plans over a small operator algebra, state what a quality contract obliges a service to do, and then distinguish two ways of discharging that obligation – by inference, which is what the literature does, and by accounting, which is what we propose.

3.1. Hybrid RAG plans over heterogeneous knowledge sources

Let $\mathcal{S}$ be a set of evidence sources: relational tables, text corpora with sparse and dense indexes, and remote structured knowledge APIs. A *plan* p is a directed acyclic graph built from typed operators

$$\mathsf{Retrieve}_{s,k} \mid \mathsf{Fuse} \mid \mathsf{Rerank}_m \mid \mathsf{Filter}_\phi \mid \mathsf{Generate}_g,$$

where $\mathsf{Retrieve}_{s,k}$ fetches k evidence units from source s (SQL row selection, BM25, dense approximate nearest neighbour, or API calls), Fuse merges rankings, Rerank_m re-scores with a cross-encoder m, Filter_ϕ drops evidence failing a predicate ϕ, and $\mathsf{Generate}_g$ produces the answer with generator g. Executing p on a query q yields an answer, an evidence set with a citation map, and a realised cost metered at run time.

The plan space is the Cartesian product of an *access path* – the retrieval-side configuration, which is shared across generators once materialised – and a *generator*, which may be a hosted commercial model or a locally served open-weight one. Systems in the literature almost universally restrict attention to a diagonal of this product, an escalation ladder in which retrieval depth and generator tier increase together. Section 6 shows the diagonal misses the plans a strict contract needs, and also that enlarging the space is not by itself an improvement, because it aggravates the multiplicity problem that the incumbent certification machinery has.

3.2. Evidence contracts

Definition 1 (Evidence contract). *An* evidence contract *is a pair* $\langle \ell, \alpha \rangle$ *where* $\ell : (a, q) \mapsto [0, 1]$ *is a bounded, auditable violation loss and* $\alpha \in (0, 1)$ *is the admissible violation rate. Examples of* ℓ *are* $\mathbf{1}[\text{quality}(\cdot) < \tau_q]$, $\mathbf{1}[\text{citation recall}(\cdot) < \tau_c]$, $\mathbf{1}[\max_e \text{age}(e) > \Delta_f]$, *and* $\mathbf{1}[\text{latency}(\cdot) > B_\ell]$.

A financial-services assistant is the running example: at most 10% of answers incorrect, at most 10% carrying an unsupported claim, at minimal spend. The requirement is on the *service*, over a stream of queries, not on any single answer – which is what makes it a rate and what will let us enforce it by bookkeeping.

3.3. The action space, and the action the literature omits

The executor's action set is $\mathcal{A} = \mathcal{P} \cup \{\perp\}$, where $\mathcal{P}$ is the plan space and $\perp$ is the decision to *decline*: return "insufficient evidence", route the query to a human, or degrade to a document list. Declining produces no answer and therefore no incorrect answer, so

$$\ell(\perp, q) = 0 \quad \text{for every } q, \qquad c(\perp, q) = c_\perp, \tag{1}$$

where $c_\perp$ prices the handoff. We treat $c_\perp$ as a parameter of the deployment rather than fixing it, and sweep it, because its value is an operational fact about the service and not something a paper can assert: a consumer search box degrades cheaply, a clinical decision support system does not.

Omitting $\perp$ is the single modelling decision that makes strict contracts unreachable. With $\mathcal{P}$ alone the attainable risk is bounded below by $\min_p r_p$, so a contract stricter than the best available pipeline is infeasible by construction rather than by statistics – and $\min_p r_p$ is 0.59 on CRAG and QAMPARI, 0.27 on HybridQA and 0.12 on ASQA, so the 10% contract of our running example is unexpressible on all four. Abstention is standard in selective prediction [7, 18] and in conformal methods, but it is treated there as a property of a single predictor; here it is an *action* the optimizer costs and schedules alongside the plans, which is what makes the risk–cost frontier continuous.

3.4. Two ways to discharge the obligation

Let $R(\pi) = \mathbb{E}_q[\ell(\pi, q)]$ and $\text{cost}(\pi) = \mathbb{E}_q[c(\pi, q)]$. The design goal is

$$\min_\pi \ \text{cost}(\pi) \quad \text{s.t.} \quad R(\pi) \le \alpha, \tag{2}$$

with $\mathcal{D}$ unknown. Since R cannot be computed, the obligation must be discharged some other way, and there are exactly two.

Definition 2 (Certified deployment). *A procedure mapping a calibration sample* $Z_{1:n}$ *to a policy* $\hat{\pi}$ *is* (α, δ)-certified *if* $\mathbb{P}_{Z_{1:n}}(R(\hat{\pi}) \le \alpha) \ge 1 - \delta$*, where the probability is over the calibration draw and the guarantee is about the population* $\mathcal{D}$ *that generated it.*

Definition 3 (Contract-enforcing execution). *An executor is* contract-enforcing *if the actions* $a_1, a_2, \ldots$ *it emits on a query stream* $q_1, q_2, \ldots$ *satisfy*

$$\textstyle\sum_{s \le t} \ell(a_s, q_s) \ \le \ \alpha t, \quad \text{all } t \ge 1,$$

for every query sequence, with no probability attached.

The two are not stronger and weaker versions of one statement; they differ in what they are about. Definition 2 concerns an unobservable population mean and is conditional on the deployment traffic being exchangeable with the calibration sample. Definition 3 concerns the realised trajectory – the thing an auditor would actually measure – and is conditional on nothing. It is also the property

an operator asked to sign a service-level agreement needs, because an SLA is checked on the log, not on a hypothetical population.

Existing RAG optimizers provide neither. Workload-level optimizers such as Abacus [27] compare point estimates against the constraint; adaptive routers [26] make no feasibility statement. Conformal RAG methods [3, 4] achieve Definition 2 for a single fixed pipeline, leaving the cost-minimisation dimension of (2) untouched, and Learn-then-Test [1] achieves it for a searched policy at a multiplicity price. This paper achieves Definition 3, and recovers the cost objective as a separate, purely statistical problem in which errors are paid for in money.

Remark 1 (What the guarantee is about). A contract is stated with respect to a loss the operator can audit. Both definitions apply to whatever ℓ is declared – token-F1 against gold answers, NLI-based citation entailment, an LLM judge following a benchmark's official protocol, evidence age, wall-clock latency. Neither establishes that the declared predicate captures the application's intent, and neither promises per-query correctness. The deterministic guarantee additionally requires the loss to be observed, which costs money; Section 4.4 prices observing only part of it, and Section 7 discusses a systematically biased auditor, which no accounting scheme can repair.

4. Enforcing a contract by accounting

This section develops the mechanism. Section 4.1 states the ledger gate and proves that it makes the contract unbreakable; Section 4.2 characterises what becomes attainable once declining is an action; Section 4.3 gives the optimisation layer and its regret; Section 4.4 prices partial labelling; Section 4.5 compares the concession this design makes with the one a calibration certificate makes.

4.1. The ledger gate

Fix a contract level $\alpha \in (0,1)$ and write $\ell_t \in [0,1]$ for the violation loss incurred on the t-th arrival. Define the *ledger*

$$B_t \;=\; \alpha t - V_t, \qquad V_t \;=\; \sum_{s\le t} \ell_s, \qquad B_0 = 0, \tag{3}$$

the contract allowance earned but not yet spent. The controller is the following rule, and nothing else is needed for validity.

Definition 4 (Ledger gate). *At arrival t the executor may run any plan $p \in \mathcal{P}$ if $B_{t-1} \ge 1-\alpha$; otherwise it must take the decline action $\perp$, for which $\ell_t = 0$ by definition.*

Theorem 1 (Pathwise contract satisfaction). *Under Definition 4, $B_t \ge 0$ and hence*

$$V_t \;\le\; \alpha t \qquad \text{for every } t \ge 1, \tag{4}$$

on every sample path, for every query sequence, whatever plan the executor chooses when the gate is open.

PROOF. Induction on t. $B_0 = 0 \ge 0$. Suppose $B_{t-1} \ge 0$. If the gate is open then $B_{t-1} \ge 1-\alpha$, and since $\ell_t \le 1$,

$$B_t = B_{t-1} + \alpha - \ell_t \;\ge\; (1-\alpha) + \alpha - 1 = 0.$$

If the gate is closed the executor declines, $\ell_t = 0$, and $B_t = B_{t-1} + \alpha \ge B_{t-1} \ge 0$. In both cases $B_t \ge 0$, which is (4).

Three things are absent from this argument and their absence is the contribution. There is no probability: (4) is not an event of probability $1-\delta$ but an identity of the realised trajectory. There is no distributional assumption: the queries may be adversarial, adaptively chosen against the executor, or drawn from a distribution that changes at every step, and the bound is unaffected – so drift needs no detector, no restart schedule and no error budget, because there is no assumption for drift to falsify. And there is no dependence on $|\mathcal{P}|$: the executor may consider a hundred plans or a million, and pays nothing for having considered them, which is exactly the multiplicity price a calibration certificate cannot avoid.

What Theorem 1 does not do is make the system useful. It would be satisfied by declining every query. The content of the design is that validity has been removed from the optimisation problem entirely, so the optimiser can be judged on cost alone – and, being unable to cause a violation, can be far more aggressive than a certifier is allowed to be. The rest of this section is about cost.

Remark 2 (Bounded losses). The gate uses $\ell_t \le 1$ to size the reserve $1-\alpha$. For a bounded loss $\ell_t \le L$ the same argument gives the gate $B_{t-1} \ge L-\alpha$; for an unbounded loss no deterministic reserve exists and the guarantee necessarily becomes probabilistic. Contract losses in this setting are indicators of an audit predicate (Section 3) and are bounded by construction.

4.2. What declining makes attainable

Let $r_p = \mathbb{E}[\ell(p,q)]$ and $c_p = \mathbb{E}[c(p,q)]$ be the risk and cost of plan p under the query distribution, and let the decline action have risk 0 and cost $c_\perp$. A randomised policy is a distribution w over $\mathcal{P} \cup \{\perp\}$, and the offline problem is the linear program

$$\mathrm{OPT}(\alpha) \;=\; \min_{w\in\Delta} \sum_a w_a c_a \quad \text{s.t.} \quad \sum_a w_a r_a \;\le\; \alpha. \tag{5}$$

Proposition 1 (Feasibility and its price). *Without the decline action, (5) is feasible if and only if $\alpha \ge \min_p r_p$. With it, (5) is feasible for every $\alpha > 0$, and if $\alpha < \min_p r_p$ every feasible policy declines at least a fraction $1 - \alpha/\min_p r_p$ of arrivals. Moreover an optimal solution of (5) has at most two nonzero weights.*

PROOF. The attainable risks of the plan-only simplex form the interval $[\min_p r_p, \max_p r_p]$, giving the first claim. Declining everything has risk 0, so the feasible set is nonempty for any $\alpha > 0$. If $\sum_a w_a r_a \le \alpha$ and $r_a \ge \min_p r_p$ for every $a \ne \perp$, then $(1-w_\perp)\min_p r_p \le \alpha$, i.e. $w_\perp \ge 1 - \alpha/\min_p r_p$. The last claim is the standard basic-solution bound: (5) has two constraints, so a vertex has at most two nonzero coordinates.

Algorithm 1 Ledger-gated execution

$B \leftarrow 0$; $\hat{r}, \hat{c}$ from history; $n \leftarrow n_0$
for each arriving query q_t **do**
  if $B \geq 1 - \alpha$ **then**
    $w \leftarrow$ solve (5) with $\tilde{r}$ from (6) (refreshed periodically)
    draw action $a \sim w$
  else
    $a \leftarrow \perp$ {gate closed}
  end if
  execute a; pay c_a; observe loss ℓ_t if audited
  $B \leftarrow B + \alpha - \ell_t$
  update $\hat{c}_a$ always, $\hat{r}_a$ if audited
end for

Proposition 1 is why declining cannot be dismissed as a degenerate escape. Below $\min_p r_p$ the contract is unattainable by any pipeline no matter how much is spent, so an optimizer that must answer every query has nothing to return; the honest response is to report how much traffic has to be routed elsewhere, and at what price. This is the regime in which strict contracts live – $\min_p r_p$ is 0.59 on two of our four tasks – and it is invisible to the existing formulation. The two-nonzero-weight property matters operationally: the deployed policy is always "run plan i, or decline, with probability w", never an opaque dense mixture.

4.3. The optimisation layer and its regret

The risks r_a are unknown, so the executor maintains an empirical mean $\hat{r}_a$ over the labelled arrivals it has served with a, a count n_a, and the mean observed cost $\hat{c}_a$. Costs and risks are observed on different schedules and conflating them is a modelling error: the price of a call is on the invoice when the call returns, so $\hat{c}_a$ advances on every execution, whereas whether the answer was acceptable requires an auditor and $\hat{r}_a$ advances only on labelled ones. Every `resolve_every` arrivals the executor solves (5) with r_a replaced by

$$\tilde{r}_a \;=\; \hat{r}_a + \kappa\sqrt{1/(4n_a)} \tag{6}$$

and samples its action from the optimal w. Algorithm 1 is the whole controller.

The margin $\kappa/\sqrt{4n_a}$ in (6) deserves comment because it resembles the object this paper argues against. Aiming the mixture exactly at α is a mistake the ledger makes visible: estimation error is two-sided, so a policy placed on α sits above it half the time, the ledger drifts down, the gate closes and the arrival is declined – at $c_\perp$, the most expensive outcome available. Steering slightly below α gives the ledger positive drift and the gate stops firing. The margin is therefore a cost-versus-cost device, and it differs from a confidence width in the two ways that matter: it carries no part of the guarantee, so κ is an $O(1)$ constant rather than $\sqrt{\log(K/\delta)}$ over a candidate set, and its denominator is the number of arrivals served so far, which grows without bound, so it decays to zero over the deployment. Section 4.5 quantifies both differences on our data, and the experiments sweep κ over $[0, 4]$: cost moves by 6–112% across the sweep while the breach count stays at zero throughout, which is what "the constant trades cost against cost" means operationally.

Theorem 2 (Cost regret). *Assume the stream is i.i.d. with plan risks r_a and costs bounded by $c_{\max}$, and let $\bar{c}_T$ be the mean serving cost of Algorithm 1 over T arrivals. Then*

$$\mathbb{E}[\bar{c}_T] - \mathrm{OPT}(\alpha) \;=\; O\!\left(c_{\max}\sqrt{\frac{P\log T}{T}}\right), \tag{7}$$

where $P = |\mathcal{P}|$.

PROOF SKETCH. The proof has two parts.

Estimation. Set $\kappa = \Theta(\sqrt{\log T})$. The bounds $\tilde{r}_a$ dominate every r_a at every $t \leq T$ with probability $1 - O(T^{-1})$, by Hoeffding and a union bound over P plans and T steps; on that event the LP solved by the executor is a restriction of (5), so its value exceeds $\mathrm{OPT}(\alpha)$ by at most the sensitivity of the LP to tightening the constraint by $\max_a \kappa/\sqrt{4n_a}$. The value function of (5) is piecewise linear and convex in α with slope bounded by $c_{\max}/\min_{a\neq\perp} r_a$, giving a per-step excess $O(c_{\max}\kappa/\sqrt{n_{a_t}})$. Summing over t and using $\sum_t n_{a_t}^{-1/2} = O(\sqrt{PT})$ yields the stated rate. *Gate closures.* While the estimates are accurate the deployed mixture has risk at most $\alpha - \Omega(\kappa/\sqrt{n})$, so B_t is a random walk with positive drift; the expected number of arrivals at which it is below the reserve $1 - \alpha$ is $O(\sqrt{T})$ by the reflection principle, each costing at most $c_{\max}$. Both terms are $O(c_{\max}\sqrt{PT\log T})$ in total.

Note what Theorem 2 is and is not. It is an efficiency statement and it does assume an i.i.d. stream; if that assumption fails the cost degrades and the experiments in Section 6 measure by how much under four non-exchangeable orderings. Theorem 1 keeps holding throughout. Separating the two – assumptions for efficiency, none for safety – is the structural difference from a calibration certificate, where the same exchangeability assumption carries both.

4.4. Paying for labels

Theorem 1 consumes ℓ_t for every served query, which in production means an auditor – an LLM judge, or a human on sampled traffic. Suppose only a fraction ρ of served arrivals is labelled. Two accounting rules are available and they differ in kind.

Proposition 2 (Partial labelling). *If unlabelled arrivals are charged $\ell_t = 1$ (*worst-case *accounting), (4) continues to hold pathwise. If instead they are charged 0 and labelled ones are charged ℓ_t/ρ (*inverse-probability *accounting), the ledger is an unbiased estimate of $\alpha t - V_t$ and $V_t \leq \alpha t + O(\sqrt{t\log(1/\delta)/\rho})$ holds with probability $1 - \delta$ by Ville's inequality, but (4) does not hold pathwise.*

The first rule keeps a guarantee and pays for it in declines: an unlabelled query consumes allowance as though it had failed, so the gate closes sooner. The second keeps

the cost and gives up the guarantee. We report both, and the measured gap is not academic – at $\rho = 0.25$, worst-case accounting never breaches while inverse-probability accounting breaches on 6% (ASQA) to 59% (HybridQA) of streams, at essentially the same cost (Section 6). Auditing is not free and this paper does not pretend otherwise; the audit bill is carried in every cost figure we report, and on the cheapest contracts it is the dominant term.

4.5. Two concessions compared

Both designs give up something to be safe. A fixed-sequence certifier concedes a confidence half-width $w_{\text{stat}} = \sqrt{\log(2K/\delta)/(2n_{\text{cal}})}$: it deploys a policy whose estimated risk is at most $\alpha - w_{\text{stat}}$, and since the decision is never revisited, that slack is paid on every query for the life of the deployment. The ledger concedes the reserve $1-\alpha$ and the margin $\kappa/\sqrt{4n_a(t)}$. The reserve is a constant, so its per-query cost $(1-\alpha)/t$ vanishes; the margin shrinks as the deployment serves traffic.

The gap is large on real numbers. With $K = 69$ candidates, $\delta = 0.1$ and $n_{\text{cal}} = 500$, $w_{\text{stat}} = 0.085$; the ledger's margin after a stream of 6 466 HybridQA queries concentrated on two or three plans is $\kappa/\sqrt{4n} = 0.011$ at $\kappa = 1$ – 7.9× tighter, and 2.8–4.4× tighter on the shorter streams – and 0 at $\kappa = 0$, where validity is unchanged. On a contract at $\alpha = 0.20$ the certifier is therefore steering at 0.115 while the ledger steers at 0.189. It is not that the certifier is insufficiently clever; it is aiming at a different and much stricter target, and buying the plans that hit it. Table 1 shows the consequence directly: the certified policies realise risks far below the contract they were given, which is wasted allowance paid for in expensive plans and unnecessary declines.

5. Experimental setup

5.1. Research questions

- **RQ1 (does the contract hold?)** Over served streams, does the realised violation rate stay below α at *every* prefix, for the ledger and for a one-shot certifier, and what happens to each when the stream is not exchangeable with the calibration prefix?
- **RQ2 (what does it cost?)** How much does each method spend per query – serving, declining and auditing – relative to an offline optimum that already knows every plan's risk?
- **RQ3 (which ingredient matters?)** Declining, randomisation and the ledger are three separable changes. What does each contribute, and in particular, how much of the gain is explained by declining alone?
- **RQ4 (is it efficient, and how fast?)** Does the cost gap to the offline optimum decay at the $O(T^{-1/2})$ rate of Theorem 2, and how sensitive is the controller to its one constant κ?
- **RQ5 (what do labels cost?)** Auditing every served query is what makes the guarantee deterministic. What is the price of auditing less, under each of the two accounting rules?
- **RQ6 (does the plan space help?)** Unbundling access path from generator gives a 36-plan grid on HybridQA and a 24-plan grid on CRAG. Does a larger space help a static certifier, does it hurt it, and can an operator tell which in advance?

5.2. Datasets and contracts

Four heterogeneous benchmarks, all executions real and metered. (A) **HybridQA** [32]: multi-hop QA over Wikipedia tables joined with entity passages; sources are a structured row-selection operator and sparse/dense passage indexes over 440K passages; loss $\mathbf{1}$[token-F1 $<$ 0.5]; history/stream = 5000/6466. (B) **CRAG** [33]: web QA with the official mock knowledge-graph API over five domains; answers graded by an LLM judge against the gold answer following CRAG's protocol; loss $\mathbf{1}$[not correct]; history/stream = 700/2006. (C) **ASQA** [34] under the official ALCE evaluation [35]: long-form answers with citations over a pooled GTR corpus, citation recall scored by the official TRUE-NLI entailment model [36]; loss $\mathbf{1}$[citation recall $<$ 50]; history/stream = 150/798. (D) **QAMPARI** [37] under the same protocol: list answers over an 86K-passage corpus; same loss; history/stream = 150/850.

The four span the range that matters for this paper. The risk floor $\min_p r_p$ is 0.12 on ASQA, 0.27 on HybridQA and 0.59 on both CRAG and QAMPARI, so contract levels are chosen per task to straddle that floor: levels below it are unattainable without declining and are marked as such throughout. Stream lengths differ by a factor of eight, which is what lets us separate the asymptotic behaviour of the controller from its transient.

5.3. Plans, the plan grid, and cost metering

The base plan space per benchmark is a four-rung cost-ascending ladder: from sparse-only retrieval with a flash-tier generator up to hybrid reciprocal-rank-fusion retrieval, cross-encoder reranking, NLI verification, knowledge-graph-API evidence (CRAG), citation repair (ASQA, QAMPARI) and a max-tier generator. Adding the per-query routing and mixture candidates of Section 4.3 gives 69 candidate policies, of which the Pareto frontier on historical data retains 10–15; both methods are given the same frontier, so the comparison is not about screening.

The plan grid (RQ6) The ladder is a hand-drawn diagonal through a larger space: retrieval strength and generator tier rise together, so it never contains a plan that retrieves deeply and generates cheaply. We therefore also executed the full Cartesian product of access paths and generators on two benchmarks. On HybridQA the product is four access paths × nine generators: six hosted, spanning three model developers and three price tiers (Qwen flash/plus/max, DeepSeek

v4-flash/v3.2, GLM 4.7), and three open-weight models served locally (Gemma 3 4b, Gemma 3 12b, Llama 3.1 8B), giving 36 physical plans, Pareto pruned to 8 on the training split. On CRAG the six hosted generators give 24 plans, pruned to 5, with answers labelled by the same judge as the CRAG ladder so that the grid and the ladder are scored identically. Each access path's retrieval view is materialised once and shared by every generator, so retrieval operators are paid once, which is the RAG analogue of a materialised common subexpression; only generation is metered per plan. Reporting a second grid matters because the effect of enlarging the space turns out to have a task-dependent sign (§6.6), which a single benchmark would have hidden.

Metering The physical layer uses DuckDB [38] for structured sources, BM25 [39] for sparse retrieval, BGE embeddings [40] with FAISS [41] for dense retrieval, reciprocal-rank fusion [42], a bge-reranker-v2-m3 cross-encoder and DeBERTa-based NLI verification; local generators run on two RTX 4090 GPUs. Hosted calls are token-metered at each developer's list prices; local calls are metered in GPU-seconds at an RTX-4090 rental rate, so hosted and local plans share one currency. All costs are milli-CNY (mCNY) per query. Every LLM and API call passes through a thread-safe cache keyed by (model, messages, decoding parameters), so every table regenerates from the artifact with no further calls.

The price of declining $c_\perp$ is a property of the deployment, not of the method, so we sweep it at $\{0.5, 1, 2\}\times$ the cost of the most expensive plan in the space. All tables report the middle setting unless stated; the sweep is in Section 6.2. Pricing a decline at the strongest plan's cost is the conservative reading: it says a human handoff costs about what the best pipeline costs, so the optimiser gets no free lunch from declining.

5.4. Protocol and baselines

The stream The pooled calibration and test executions form a finite population; one *draw* is an ordering of it. Both systems serve the same stream and are charged for everything they consume. This is the point where the usual comparison is unfair to no one but the reader: a one-shot certifier looks cheap only if the queries it burns on calibration are free, and they are not – they are real queries, served without a certificate, by whatever plan the history liked. We therefore charge the certifier's calibration prefix at the historically safest plan and charge its labels, exactly as we charge the ledger's.

Baselines *LTT, no decline* is fixed-sequence Learn-then-Test [1] with Hoeffding–Bentkus p-values over the 69 candidates, $n_{\text{cal}} = 500$, $\delta = 0.1$ – the incumbent formulation and the method of our own earlier design. *LTT + decline* is the same procedure with $\perp$ added to the candidate space, including mixtures of a plan with $\perp$ on a 9-point grid; this is the baseline that isolates our contribution from the mere availability of declining. *Ledger, deterministic plan* is Algorithm 1 restricted to single plans, which can reach only frontier vertices. *Offline action LP* solves (5) on the population risks and costs: it knows what no deployed system knows and buys no labels, so it is a lower bound rather than a competitor, and it is what "× oracle" means throughout.

Reported quantities For each configuration we run 20–30 independent streams and report the mean per-query cost (serving + declines + audit), the realised risk over the whole stream, the decline rate, the worst rate the run ever reached, $\sup_t V_t/t$ over $t \geq 200$, and *PathV* – the number of streams on which that worst rate exceeded α. All rates are reported in their own units and read against the α that was asked for, rather than normalised by it. The worst rate is the quantity Definition 3 controls and that Definition 2 says nothing about, and PathV is the count that a pathwise guarantee, as opposed to one that holds with probability $1 - \delta$, is entitled to claim is zero. The first 200 arrivals are excluded because no method can control a rate over a handful of queries, and the exclusion is applied identically to both. Costs are additionally reported relative to the offline optimum, which we abbreviate OPT.

6. Results

6.1. Does the contract hold? (RQ1)

Table 1 reports the comparison at each task's strictest non-degenerate contract α^*. The column that carries the claim is *PathV*, the number of streams whose running rate exceeded the contract at any point: the ledger is $0/30$ on all four tasks, and by Theorem 1 this is an identity rather than an outcome that 30 draws happened to produce. What makes it worth reporting anyway is the column beside it. The worst rate the executor ever reached is 0.297 against a contract of 0.30, 0.607 against 0.62, 0.138 against 0.15 and 0.583 against 0.60 – between 92 and 99 percent of the allowance, spent and not crossed. A guarantee that never binds would be worth little; this one binds within one to eight percent everywhere, and the loosest figure belongs to ASQA, the shortest stream in the suite at 798 queries, where the plan estimates are still in their transient.

The certifier separates into two failure modes that the same table shows side by side. Without the decline action it breaches 2 to 21 streams in 30, because α^* sits only a few points above the risk floor and its confidence width is wider than that margin, so the policy it certifies is the safest plan and that plan's risk is not below the contract. Handing it the decline action removes every breach, and the *Risk* column shows what that costs: realised risks of 0.202, 0.317, 0.035 and 0.131 against contracts of 0.30, 0.62, 0.15 and 0.60. It is safe because it aims at $\alpha - w_{\text{stat}}$ and then, for want of a plan there, refuses. Its bill barely moves – 6.97 against 7.11 mCNY on HybridQA, 7.38 against 7.35 on QAMPARI – which is the cleanest statement of the problem we are attacking: the decline action buys a certifier validity and buys it nothing else, because the confidence width has to be paid whether it is spent on a plan or on refusing. The ledger pays 1.27 to 1.75× the offline optimum where the certifier with the same action space pays 2.89 to 5.85×.

Table 1
Contract adherence and cost of ownership at each task's strictest non-degenerate contract α^*, the first level at or above the risk floor $\min_p r_p$. *LTT* is one-shot certification, *LTT + decline* the same certifier handed the decline action, *Ledger, greedy* the deterministic ablation. All see the same plan space, the same decline price and the same stream, and all are charged for calibration, service, declines and labels. *Risk* is the realised rate over the stream and $\sup_t$ the worst it reached at any point after $t = 200$; both are to be read against the α^* in the block heading. *PathV* counts the streams, of 30, whose running rate exceeded α^* at any time – the quantity a pathwise guarantee speaks to, and 0/30 for the ledger by Theorem 1 rather than by luck. *OPT* is the offline optimum, which knows every plan's risk in advance and is not implementable. Best implementable cost among methods that did not breach is in bold. Costs are mCNY per query.

Method	Risk	$\sup_t$	PathV	Decl.%	Cost	Cost/OPT
HybridQA ($\alpha^* = 0.30$, $\min_p r_p = 0.268$, $N = 6466$)						
LTT	0.273	0.292	10/30	0.0	7.11	4.78
LTT + decline	0.202	0.203	0/30	22.5	6.97	4.68
Ledger, greedy	0.000	0.000	0/30	100.0	7.23	4.86
Ledger	0.291	0.297	0/30	14.8	**1.90**	1.27
Offline OPT	$\leq \alpha^*$	–	–	–	*1.49*	*1.00*
CRAG ($\alpha^* = 0.62$, $\min_p r_p = 0.591$, $N = 2006$)						
LTT	0.592	0.623	13/30	0.0	0.78	3.37
LTT + decline	0.317	0.317	0/30	21.8	0.79	3.38
Ledger, greedy	0.000	0.000	0/30	100.0	0.79	3.39
Ledger	0.600	0.607	0/30	9.0	**0.41**	1.75
Offline OPT	$\leq \alpha^*$	–	–	–	*0.23*	*1.00*
ASQA ($\alpha^* = 0.15$, $\min_p r_p = 0.120$, $N = 798$)						
LTT	0.121	0.136	2/30	0.0	7.79	2.86
LTT + decline	0.035	0.035	0/30	10.1	7.87	2.89
Ledger, greedy	0.000	0.000	0/30	100.0	7.69	2.82
Ledger	0.129	0.138	0/30	38.5	**3.77**	1.38
Offline OPT	$\leq \alpha^*$	–	–	–	*2.73*	*1.00*
QAMPARI ($\alpha^* = 0.60$, $\min_p r_p = 0.591$, $N = 850$)						
LTT	0.591	0.614	21/30	0.0	7.35	5.83
LTT + decline	0.131	0.131	0/30	18.9	7.38	5.85
Ledger, greedy	0.000	0.000	0/30	100.0	7.23	5.73
Ledger	0.573	0.583	0/30	15.9	**1.74**	1.38
Offline OPT	$\leq \alpha^*$	–	–	–	*1.26*	*1.00*

Over the whole contract range One contract level per task is a thin basis for a claim about a mechanism, so Table 2 sweeps α over 15 levels per task and Figure 1 plots all of them. The picture is the standard validity diagram of this literature – realised rate against the contract that was asked for, with the region above the diagonal forbidden – and it separates the three mechanisms cleanly. The ledger tracks the diagonal from beneath across the entire range on all four tasks, honouring 15 levels of 15 everywhere including every level below the risk floor. The certifier without a decline action sits above the diagonal wherever the contract is stricter than what its safest plan delivers, breaching 545 of the 1 200 streams swept and honouring as few as 3 levels of 15 on QAMPARI. Given the decline action it drops well below the diagonal instead, which is the visual signature of unspent allowance: valid, and conservative by a margin it pays for. Its mean cost over the sweep is 5.49× the offline optimum against 5.47× without the decline action, while the ledger averages 1.57× and never exceeds 2.58× at any level of any task.

When the stream is not exchangeable Table 3 is the experiment that separates the two guarantees rather than merely comparing their prices. A calibration certificate is valid under exchangeability between the calibration prefix and what follows; we construct four orderings that break it. Reporting adherence and price side by side is what makes the result legible, because the certificate fails in both directions and the two failures look nothing alike. Under *easy-first* – the population sorted so that queries most plans answer correctly arrive first – the certifier calibrates on an unrepresentative prefix and certifies a cheap plan. It then pays as little as 0.30 of the offline optimum and overruns its contract on three tasks in four, by 1.25 to 2.18×; *mid-stream drift* does the same thing more mildly. Under *hard-first* and *adversarial* the prefix is pessimistic instead, so the certifier retreats to its safest plan, declines 33–92% of the traffic, and lands at 2.9 to 7.3× the optimum. Those rows record no violations at all, which is worth naming rather than glossing: a stream that is mostly refused and otherwise answered by the strongest plan cannot accrue any. It is the arithmetic of a system that has stopped doing its job, not evidence that the certificate works.

A certificate therefore fails unsafely in one direction and expensively in the other, and has no setting in which it is merely correct – which is uncomfortable precisely because nothing tells the operator which direction they are in. The ledger has one behaviour instead of two: it reaches 85 to 100% of its allowance on every row without ever passing it, and pays 1.00 to 2.77× the optimum. Reordering does move that price – on HybridQA the same contract costs 1.06× the optimum under easy-first and 1.34× under hard-first – but it does not move the guarantee at all, which is the separation Theorem 2 predicts: non-exchangeability is a cost problem here, not a safety problem.

Is the certifier simply given too generous a budget? The natural defence of the certificate is that δ was set too high, and that a stricter budget would restore validity. Table 4 tests it by sweeping δ from 0.2 to 0.001, a factor of 200 that widens the confidence correction by a third. Across 2 400 certified streams it changes the breach count in one of twelve cells. On HybridQA and its plan grid all 100 streams per cell breach at both ends of the sweep, with realised risk 2.1–2.4 times the contract; on CRAG, QAMPARI and the CRAG grid the counts are identical at $\delta = 0.2$ and $\delta = 0.001$, at 80, 60, 20 and 40 streams per hundred. Only ASQA under drift responds, falling from 73 to 43 – still four streams in ten, at a budget no practitioner would choose. The reason is structural: δ prices the sampling noise in a calibration estimate, whereas the failure here is that the calibration sample describes a different population from the one served. Widening a correction for the wrong quantity does not help.

The same table answers the reader who suspects the ledger's clean record of being slack rather than control. In

Table 2
Cost of ownership across the contract range, in units of the offline optimum at each level. α is swept over 15 levels from 0.10 to 0.80 and six are shown. A superscript gives the number of streams, out of 20, whose running rate exceeded α at any point; a cell without one was honoured by all 20. *hon.* counts the levels of 15 honoured on every stream and $\alpha_{\min}$ is the strictest of them. Every task's floor exceeds 0.10 (†), so the left of the table asks for contracts no plan can meet. *LTT* cannot decline, has nothing feasible to certify there, and breaches 545 of the 1 200 streams swept. Handing it the decline action repairs validity in all 1 200 and changes its bill by almost nothing – mean 5.49× the optimum against 5.47× – because a certifier pays the same confidence width whether it spends it on a plan or on refusing. The ledger honours the same 15 levels everywhere at a mean of 1.57×, and its worst level anywhere is 2.58×, below the certifier's best on two of the four tasks. Read the strict end of each row: that is where a contract binds and where the mechanisms separate.

		cost / offline optimum at $\alpha =$						honoured	
Task	Method	0.10	0.20	0.30	0.45	0.60	0.75	hon.	$\alpha_{\min}$
HybridQA†	LTT	1.36^{20}	2.13^{20}	4.78^{5}	4.35	4.18	4.19	9/15	0.40
	LTT + decline	1.35	2.09	4.66	4.35	4.18	4.18	15/15	0.10
	Ledger	1.03	1.08	1.26	2.06	2.01	2.01	15/15	0.10
CRAG†	LTT	1.19^{20}	1.36^{20}	1.60^{20}	2.16^{20}	3.32^{18}	2.18	4/15	0.65
	LTT + decline	1.19	1.37	1.60	2.16	3.33	2.18	15/15	0.10
	Ledger	1.05	1.10	1.17	1.33	1.68	2.04	15/15	0.10
ASQA†	LTT	1.78^{20}	6.86	6.71	8.90	13.12	17.56	13/15	0.20
	LTT + decline	1.80	6.89	6.75	8.95	13.23	17.73	15/15	0.10
	Ledger	1.16	2.04	1.56	1.64	1.93	2.06	15/15	0.10
QAMPARI†	LTT	1.18^{20}	1.40^{20}	1.73^{20}	2.67^{20}	5.83^{17}	18.58	3/15	0.70
	LTT + decline	1.18	1.40	1.74	2.68	5.85	18.59	15/15	0.10
	Ledger	1.02	1.05	1.07	1.17	1.34	2.58	15/15	0.10

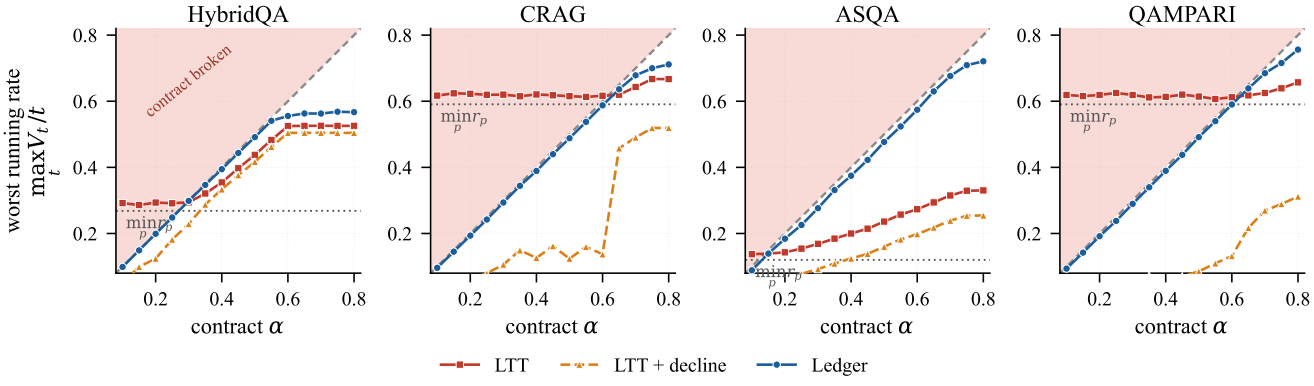

Figure 1: Worst running violation rate $\max_{t\geq 200} V_t/t$ against the contract it was asked to keep, swept over 15 levels per task, 20 streams each. Anything in the shaded region broke its contract. The dotted horizontal line is the risk floor $\min_p r_p$: to the left of where it meets the diagonal, no plan satisfies the contract and keeping it requires declining. The ledger tracks the diagonal from beneath everywhere, including far below the floor. The certifier crosses it wherever the contract is stricter than its safest plan; given the decline action it stops crossing but drops well below instead, and the distance between that curve and the diagonal is allowance it paid for and did not use.

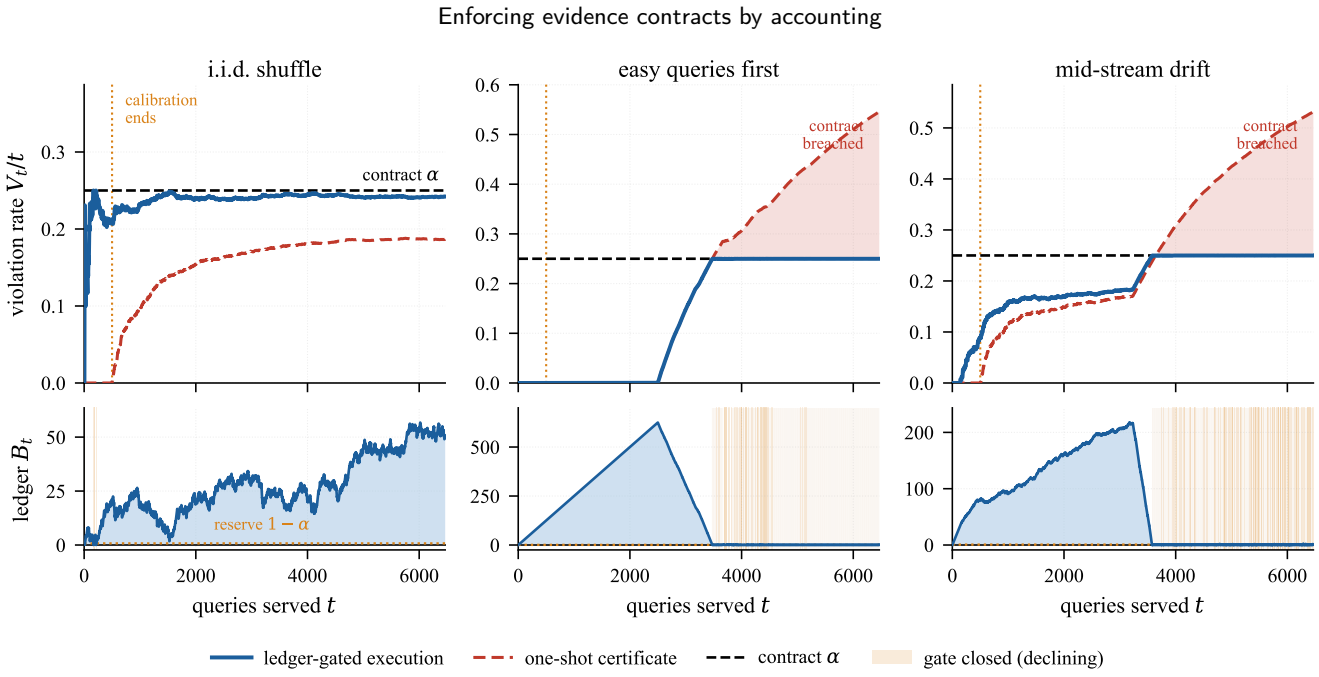

Figure 2: The gate at work on one HybridQA stream at $\alpha = 0.25$ under three orderings. *Top*: running violation rate V_t/t; the shaded area is the certificate's excess over the contract. *Bottom*: the ledger B_t that produces the behaviour above, with the reserve $1-\alpha$ and the intervals in which the gate is closed. Under an i.i.d. shuffle the ledger accumulates a surplus and the executor spends it, tracking the contract from below while the certificate leaves a third of the allowance unused. Under the two non-exchangeable orderings the certificate calibrates on an unrepresentative prefix and is breached; the ledger accumulates allowance on the easy prefix, spends it exactly, and then declines.

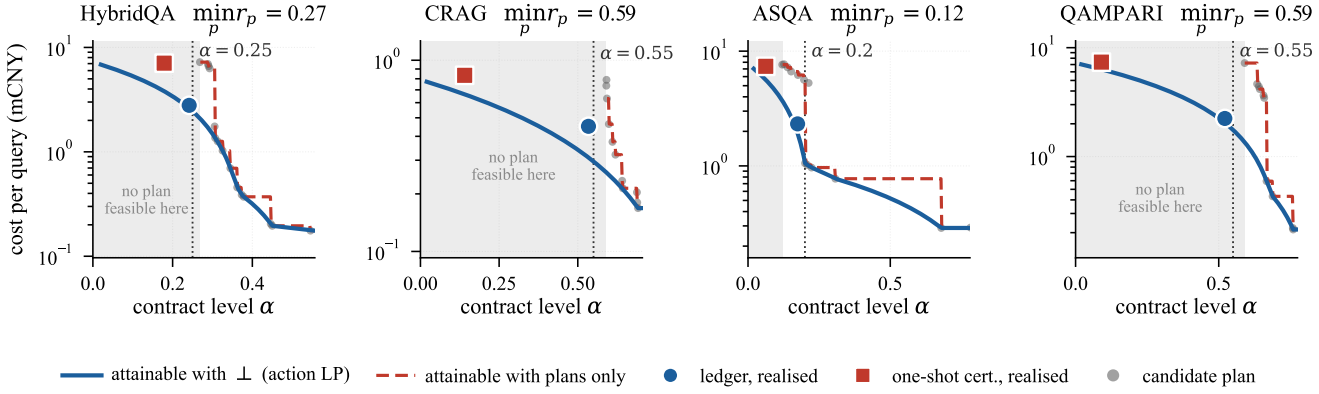

Figure 3: Cheapest attainable cost as a function of the contract level. Small dots are the individual plans; the dashed staircase is the cheapest of them that clears α, which is what a system obliged to answer every query can reach, and it stops at the risk floor $\min_p r_p$. The solid curve adds the decline action, is continuous, and extends into the shaded region where no plan is feasible. Large markers are where each method operates at the median contract level, plotted at its realised risk: both mix a plan with $\perp$, so both can sit left of the floor, but the ledger sits close to the attainable curve while the certificate sits well above and to the left of it. That displacement is unspent allowance, and it is paid for in money.

these twelve cells its realised risk is 0.991 to 1.000 of α: it spends between 99.1% and 100.0% of the allowance and still does not cross the line, which is what a binding constraint looks like. The certificate crosses the line while spending less than half its allowance in the exchangeable case and more than twice too much in the non-exchangeable one.

6.2. What does it cost? (RQ2)

Relative to the offline action LP – which knows every plan's population risk and buys no labels – Table 1 places the ledger at 1.27× (HybridQA), 1.75× (CRAG), 1.38× (ASQA) and 1.38× (QAMPARI) the offline optimum at each task's α^*. The one-shot certifier with the same action space sits at 4.68, 3.38, 2.89 and 5.85×. The ratio between the two is 1.93–4.24×. Table 2 shows that this gap is not an artefact of

Table 3
Contract adherence under query orderings that violate exchangeability. A calibration certificate is valid only if the deployment stream is exchangeable with the calibration prefix; the ledger bound holds pathwise and assumes nothing about the order. Entries are means over 20 streams at the median α of each task, given in parentheses after the task name. Adherence is the worst rate the run ever reached, to be read against the α in the task heading; cost is in units of the offline optimum for the same α, so the two halves of the table read as what was promised against what was paid. The certificate fails in both directions and never in between. When the calibration prefix is easier than the stream it under-provisions: it pays as little as 0.30 of the optimum and overruns its contract on three tasks in four, by up to 2.18×. When the prefix is harder it over-provisions: it declines 33–92% of the traffic and lands at 2.9–7.3× the optimum, which is why those rows accrue no violations at all – a stream that is mostly refused and otherwise served by the strongest plan cannot. Neither is a contract honoured at a defensible price. The ledger reaches 85–100% of its contract on every row at 1.00–2.77 of the optimum, and the rows that land exactly on α do so exactly rather than by rounding, since the gate closes as the allowance runs out, so V_t/t approaches α and stops. The orderings that break the certificate are not exotic: sorting a day's traffic by difficulty is what a queue does.

		$\sup_t V_t/t$		cost / optimum	
Task	Order	LTT	Ledger	LTT	Ledger
HybridQA (0.25)	i.i.d. shuffle	0.161	0.248	2.90	1.15
	hard first	0.000	0.250	2.96	1.34
	easy first	0.546	0.250	0.30	1.06
	drift	0.530	0.250	0.30	1.05
	adversarial	0.000	0.250	2.96	1.06
CRAG (0.55)	i.i.d. shuffle	0.195	0.543	2.82	1.53
	hard first	0.280	0.550	2.92	1.76
	easy first	0.688	0.550	1.33	1.38
	drift	0.596	0.550	1.23	1.38
	adversarial	0.289	0.550	2.91	1.72
ASQA (0.2)	i.i.d. shuffle	0.063	0.181	6.86	2.15
	hard first	0.000	0.198	7.34	2.77
	easy first	0.309	0.200	4.85	2.38
	drift	0.229	0.200	4.93	2.37
	adversarial	0.000	0.170	7.33	2.36
QAMPARI (0.55)	i.i.d. shuffle	0.089	0.535	4.19	1.25
	hard first	0.000	0.548	4.18	1.89
	easy first	0.412	0.550	2.58	1.10
	drift	0.412	0.550	2.58	1.00
	adversarial	0.000	0.545	4.18	1.69

the pivotal level: it widens as the contract tightens, because a tight contract leaves the certifier's confidence width a larger share of the allowance, and it stays of the same order across the fifteen levels swept.

The decline price sweep behaves as the mechanism predicts. Halving $c_\perp$ makes declining a more attractive action and both methods use it more, which compresses the gap (on CRAG at $\alpha = 0.45$ the ledger's advantage falls to 1.19×); doubling it makes declining something to be avoided, the optimiser buys safer plans instead, and the gap widens. The ledger's advantage is therefore largest exactly where declining is expensive – which is the regime an operator cares about, since a cheap handoff means the contract was not binding in the first place.

Table 4
Sweeping the certifier's error budget over two orders of magnitude, under the two orderings that break exchangeability. *brch* counts the breaching streams out of the 100 run per row (five α levels × 20 streams); r/α is realised risk in units of the contract, so 1.00 means the allowance was spent exactly and above 1.00 means it was overspent. Shrinking δ by 200× widens the confidence correction, which is the certificate's only lever, and it moves the count in one of the twelve cells: the failure is a calibration sample that misrepresents the stream, not a correction too small for sampling noise, and no choice of δ addresses that. The ledger breaches none of the 1 200 streams here, so that column would be twelve zeros and is omitted; what is worth reading is its realised risk, which lands on the contract rather than under it and so is not slack.

		cert. $\delta = 0.2$		cert. $\delta = 10^{-3}$		ledger
Task	Order	brch	r/α	brch	r/α	r/α
HybridQA	easy	100	2.40	100	2.40	1.000
	drift	100	2.26	100	2.22	0.999
CRAG	easy	80	1.37	80	1.37	0.997
	drift	60	1.17	60	1.17	0.996
ASQA	easy	100	1.79	100	1.78	0.995
	drift	73	1.16	43	0.99	0.994
QAMPARI	easy	20	0.83	20	0.81	0.998
	drift	20	0.83	20	0.81	0.998
HQ grid	easy	100	2.15	100	2.11	1.000
	drift	100	2.00	100	1.97	0.998
CRAG grid	easy	40	1.05	40	1.05	0.992
	drift	40	1.01	40	1.00	0.991

6.3. Which ingredient matters? (RQ3)

Table 5 is the answer to the objection this design invites, namely that the gain comes from being allowed to decline.

Handing the decline action to the one-shot certifier moves its cost from 4.78 to 4.68× oracle on HybridQA, from 3.37 to 3.38× on CRAG, and makes it slightly *worse* on ASQA (2.86 → 2.89×) and QAMPARI (5.83 → 5.85×): between −2% and +1%. What it does buy is validity – the same certifier without $\perp$ breaches 2 to 21 streams in 30 at α^* on the four ladder tasks (Table 1). Declining alone therefore buys safety and almost no money. The reason is that a certifier cannot exploit it – to certify a mixture of a plan and $\perp$ it must estimate the mixture's risk from the same calibration sample and concede the same width, so it ends up declining more than necessary rather than spending its allowance.

Removing randomisation from the ledger is equally decisive in the other direction: the deterministic variant lands at 2.79–5.73× oracle, worse than the certifier on three of the four ladder tasks, and at levels below the risk floor it degenerates to declining 100% of traffic, since no single plan clears α and a deterministic rule cannot interpolate. Only the combination – a ledger that makes validity unconditional, plus a randomised action that can sit anywhere on the risk–cost segment – reaches 1.27–1.75×, and it does so in all

Table 5
Contribution of each ingredient at the same pivotal α^* used in Table 1 (the first level at or above the risk floor), reported as cost relative to the offline action LP (lower is better). Removing the decline action makes the strictest levels infeasible, and the variant then serves its safest plan and breaches; removing randomisation leaves only frontier vertices, which cannot reach a level between two plans' risks; removing the ledger returns to a calibration certificate and reinstates its confidence width. The ordering of the four variants is the same in all six settings, including the two plan grids, where the action space is 36 and 24 executed physical plans rather than routing policies over a ladder. A superscript is the fraction of streams that variant breached; a breaching variant can undercut the offline optimum, which is not an achievement but the reason a cost figure alone cannot be compared. Only the variant denied a decline action ever breaches; once declining is available the certificate is safe but dear, and the remaining gap to the last row is what the ledger and randomisation contribute.

Variant	HybridQA	CRAG	ASQA	QAMPARI	HQ grid	CRAG grid
certificate, no decline	$4.78^{0.33}$	$3.37^{0.43}$	$2.86^{0.07}$	$5.83^{0.70}$	1.63	$7.33^{0.50}$
certificate + decline	4.68	3.38	2.89	5.85	3.18	2.43
ledger, single plan	4.86	3.39	2.82	5.73	5.02	2.79
ledger + randomised	1.27	1.75	1.38	1.38	1.45	1.56

six settings, including the two plan grids whose actions are executed physical plans rather than routing policies. The three ingredients are not additive modules; each is useless without the others, which is the sense in which this is one mechanism rather than three.

6.4. Is it efficient, and how fast? (RQ4)

Table 6 tracks the serving cost above the offline optimum as the stream lengthens. The audit bill is reported beside the regret rather than inside it, because the optimum knows every risk and buys no labels; charging it for observation would confound what learning costs with what watching costs. Over HybridQA the regret falls from 0.508 to 0.109 mCNY as T goes from 500 to 32 000, ending at 2.5% of the optimum; on CRAG it reaches 1.5%, on QAMPARI 3.2%, and on ASQA – the shortest stream, hence the least converged – 7.7%. Multiplying by $\sqrt{T}$ leaves a quantity that grows by 1.7× over a 64× range of T, so the decay is close to, and slightly slower than, the $O(T^{-1/2})$ of Theorem 2; the residual is the transient in which the gate closes while the risk estimates are still loose.

The controller has one constant. Sweeping κ over $[0, 4]$ moves the cost by 6% (CRAG), 26% (HybridQA), 49% (QAMPARI) and 112% (ASQA), and the breach count over the whole sweep is zero. This is the practical content of putting the guarantee in the executor: a badly chosen constant produces a more expensive system, never an unsafe one, so it can be tuned on a cost signal that the operator can observe. A certifier's corresponding constant is δ, which cannot be tuned this way because moving it moves the guarantee. The optimum is $\kappa \approx 0$ on the long streams and $\kappa \approx 0.25$ on the short ones, which is what the argument in Section 4.3 predicts: the margin matters while n_a is small and is pure waste once it is large.

What the bookkeeping costs The other half of "how fast" is what enforcement adds to each arrival, and Table 7 measures it against the latencies recorded in the execution matrices over 64 480 metered calls. The gate is a comparison, the ledger update is two floating-point operations, and the action program is re-solved once every 20 arrivals; together they

Table 6
Serving cost above the offline optimum as the stream lengthens. The optimum solves the action LP on the population risks, i.e. what a system that already knew every plan's risk would pay; it buys no labels, so the audit bill is reported separately rather than charged against it. A roughly constant $\text{regret} \times \sqrt{T}$ is the $O(T^{-1/2})$ rate of Theorem 2. *serve* covers service and declines. Values in mCNY per query, 20 streams, at the α in parentheses.

Task	T	serve	audit	regret	reg.$\sqrt{T}$
HybridQA (0.25)	500	3.090	0.111	0.6446	14.413
	1000	2.914	0.116	0.4694	14.843
	2000	2.798	0.119	0.3533	15.798
	4000	2.707	0.122	0.2624	16.594
	8000	2.665	0.125	0.2197	19.652
	16000	2.612	0.126	0.1672	21.153
	32000	2.603	0.125	0.1583	28.318
CRAG (0.55)	500	0.327	0.134	0.0311	0.696
	1000	0.317	0.137	0.0211	0.667
	2000	0.314	0.138	0.0179	0.803
	4000	0.311	0.139	0.0151	0.955
	8000	0.305	0.140	0.0095	0.848
	16000	0.303	0.141	0.0070	0.882
ASQA (0.2)	500	2.244	0.233	1.1726	26.220
	1000	1.872	0.249	0.7998	25.290
	2000	1.779	0.253	0.7073	31.632
	4000	1.599	0.261	0.5266	33.308
QAMPARI (0.55)	500	2.106	0.166	0.3481	7.785
	1000	2.000	0.169	0.2415	7.637
	2000	1.946	0.170	0.1876	8.390
	4000	1.912	0.172	0.1540	9.737

come to 14–49 microseconds per query against retrieval and generation that take 0.8 to 2.8 seconds. Enforcement is thus between 0.0007% and 0.0018% of the pipeline it governs. Two things follow. The contract is free in wall-clock terms, so the cost figures elsewhere in this paper – which count LLM spend, declines and labels – are the whole bill. And the mechanism does not become expensive as the plan space grows: the action LP is solved over $|\mathcal{P}| + 1$ actions with a single risk constraint, so its basic optimum has at most two nonzero weights and it costs microseconds at 69 plans.

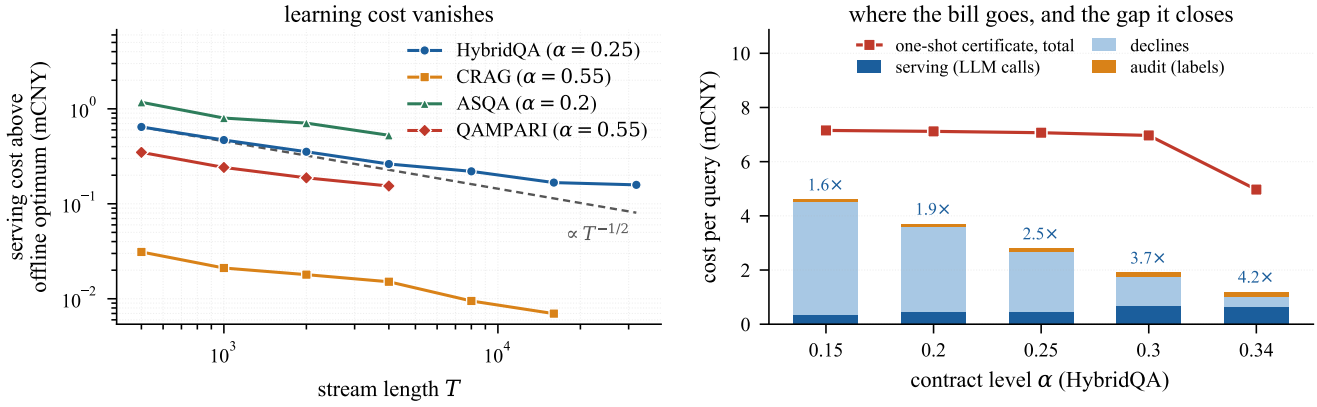

Figure 4: *Left*: serving cost above the offline optimum against stream length, log–log, at each task's median contract level; the dashed reference is $\propto T^{-1/2}$ anchored at the shortest horizon. *Right*: composition of the ledger's bill on HybridQA as the contract tightens, against the one-shot certificate's total. As α falls the executor declines more and the decline term grows, but the certificate's cost grows faster, so the gap widens from 1.6× to 4.2×.

Table 7
Run-time cost of the guarantee, microseconds per arrival. *gate* is the ledger test, *LP* the action program amortised over the 20 arrivals between re-solves, *draw* the sample from the resulting mixture and *upd.* the ledger and estimate update. Retrieval and generation are the latencies recorded in the execution matrices over 64 480 metered calls, in milliseconds. Enforcement is four to five orders of magnitude below the query it governs, so the contract is free in wall-clock terms and the cost figures elsewhere in this paper are the whole story.

		enforcement (μs)					query (ms)	
Task	$\lvert\mathcal{P}\rvert$	gate	LP	draw	upd.	total	retr.	gen.
HybridQA	69	0.11	5.1	8.7	0.50	14.4	124	674
CRAG	69	0.11	5.3	8.5	0.50	14.4	221	904
ASQA	69	0.10	39.6	8.8	0.54	49.0	283	2545
QAMPARI	69	0.11	5.3	9.8	0.51	15.7	693	1542
enforcement as a share of the pipeline: 0.0007%–0.0018%								

6.5. What do labels cost? (RQ5)

Table 8 prices the assumption that hides in Theorem 1: it consumes the loss of every served query. Under worst-case accounting an unlabelled arrival is charged a full violation, so $V_t \le \alpha t$ survives exactly – the breach count is zero at every label rate down to 10% – and the price appears as declines, which rise from 59% to 84% on HybridQA as the label rate falls from 1.0 to 0.1, taking the total cost up by 35% even as the audit line falls by 96%. Under inverse-probability accounting the cost barely moves (within 1% across the whole sweep) and the guarantee goes away: breaches appear at 6% (ASQA), 15–18% (CRAG, QAMPARI) and 59% (HybridQA) of streams at a 25% label rate.

We report this because it is the honest boundary of the contribution. A deterministic guarantee is available at full labelling, or at partial labelling for the price of declining more; it is not available for free. An operator who cannot afford either should know that the alternative on offer – an unbiased ledger – fails in practice more than half the time on our largest benchmark, which is a stronger statement against it than the theory alone makes.

6.6. Does the plan space help? (RQ6)

A query optimizer is judged partly on what happens when its search space grows. We therefore unbundled the access path from the generator on two benchmarks and executed the full product end to end: four access paths × nine generators on HybridQA (36 plans, six hosted models across three developers and three open-weight models served locally) and four access paths × six hosted generators on CRAG (24 plans, scored by the same judge as the ladder). Each grid strictly contains the hand-drawn four-rung ladder as its diagonal, so in the offline problem the larger space can only win, and does everywhere: on CRAG at $\alpha = 0.62$ the cheapest feasible mixture falls from 0.650 to 0.275 mCNY, and on HybridQA at $\alpha = 0.40$ from 0.944 to 0.641.

Certification does not inherit that monotonicity, and – the point two benchmarks let us make that one could not – does not violate it in a predictable direction either. The cleanest way to see this is to ask not what a mechanism costs at a level fixed in advance, but how strict a contract it can honour at all. Table 9 scans α on a 0.01 grid and reports the strictest level each mechanism returns anything at. Certification never reaches the risk floor: on the HybridQA ladder the best single plan attains 0.280 but nothing certifies until $\alpha = 0.31$, and on the CRAG ladder the floor is 0.579 against $\alpha_{\text{cert}} = 0.63$. That slack of 3.0 and 5.1 percentage points is a band of contracts the plan space can meet and the certificate cannot express. Counting the levels served makes the same point at scale: of the 71 levels scanned, the certifier serves 50 on the HybridQA ladder and only 18 on the CRAG ladder, while the ledger serves all 71 in both, including every level beneath the floor, because declining carries no contract loss.

That last claim is the one worth sampling rather than asserting, so we ran the ledger at 15 levels spanning $\alpha \in [0.10, 0.80]$ in all four spaces. None of the 60 cells breached,

Table 8
Price of labelling only part of the traffic. WORST-CASE charges an unlabelled arrival a full violation, which keeps $V_t \le \alpha t$ exactly; IPW rescales the labelled ones, which is unbiased but bounds the ledger only in probability. The columns price a guarantee that holds against one that usually holds. Per task: serving cost including declines, audit cost (both mCNY per query) and the worst rate the run ever reached, averaged over streams, to be read against the α in the header. That last column is the quantity the contract constrains. WORST-CASE stays under α by construction and, at low label rates, well under, because charging unlabelled arrivals as violations spends the allowance on declines instead of answers; IPW sits nearer the boundary and crosses it as labels thin out. Counted as streams breached rather than as a magnitude, the same IPW runs break the contract on 6% (ASQA) to 59% (HybridQA) of streams at a 25% label rate.

		HybridQA (α=0.25)			CRAG (α=0.55)			ASQA (α=0.2)			QAMPARI (α=0.55)		
Mode	rate	cost	audit	$\sup_t$	cost	audit	$\sup_t$	cost	audit	$\sup_t$	cost	audit	$\sup_t$
WORST-CASE	1.00	2.802	0.122	0.248	0.451	0.138	0.543	2.306	0.241	0.181	2.190	0.169	0.535
	0.50	4.861	0.033	0.147	0.450	0.056	0.455	5.524	0.047	0.076	2.878	0.071	0.454
	0.25	5.341	0.013	0.118	0.463	0.025	0.409	6.052	0.017	0.058	3.242	0.032	0.416
	0.10	5.520	0.005	0.105	0.459	0.010	0.396	6.256	0.006	0.051	3.432	0.012	0.401
IPW	1.00	2.802	0.122	0.248	0.451	0.138	0.543	2.306	0.241	0.181	2.190	0.169	0.535
	0.50	2.729	0.063	0.256	0.385	0.068	0.544	2.301	0.118	0.182	2.117	0.085	0.537
	0.25	2.781	0.030	0.252	0.358	0.033	0.530	2.222	0.060	0.182	2.190	0.041	0.530
	0.10	2.743	0.012	0.252	0.344	0.013	0.530	2.613	0.022	0.172	2.228	0.017	0.518

and the worst rate reached ranged from 71 to 99.6% of the allowance – close to the boundary where the contract binds, comfortably inside it where it does not. What the strict levels cost is visible and gradual rather than catastrophic: on the CRAG ladder, whose floor is 0.579, honouring $\alpha = 0.10$ means declining 86.5% of the stream, $\alpha = 0.30$ means 57.2% and $\alpha = 0.55$ means 18.9%, the decline rate falling smoothly as the contract loosens and the served cost tracking the offline optimum more and more tightly (1.05×, 1.17×, 1.52×) as more of the stream is actually answered. An operator who writes a contract the models cannot meet does not get a broken system or a silent violation; they get a bill for the fraction of traffic that has to go to a human, quoted before deployment.

Enlarging the space moves these quantities in opposite directions on the two tasks. On HybridQA it costs a level ($50 \to 49$) and widens the slack ($3.0 \to 4.0$ pp), because the floor does not move – the plan that attains it already lies on the diagonal – so the space contributes multiplicity and no new headroom. On CRAG it buys four levels ($18 \to 22$) and lets a contract 0.04 stricter be honoured, because the extra generators pull the floor from 0.579 to 0.536. The same reversal appears in cost at each space's own α_{cert}: the certified policy is 4.93 mCNY on the HybridQA ladder against 2.70 on its grid, and 1.60 against 1.45 on CRAG.

Neither direction is mysterious once both effects are named. A fixed-sequence procedure spends the same δ on a longer list of hypotheses, and the cheap plans a larger space adds are exactly the ones whose risks sit nearest the threshold and are hardest to certify; against that, a larger space may lower the attainable risk enough to open levels that were closed. Whether the multiplicity or the headroom dominates depends on whether the new plans move the floor, which is a fact about the models and the task. An operator learns it only after paying to build the space. That is the practical indictment: not that a certificate always gets worse when the optimizer is given more to work with, but that nobody can say beforehand whether it will.

The ledger is indifferent by construction, since its guarantee never mentions $|\mathcal{P}|$. Priced identically in the two spaces – a decline costs what it costs the incumbent deployment, so the comparison is not confounded by a cheaper fallback – at each space's own α_{cert} it sits at 1.30× the offline optimum on the HybridQA ladder against 1.43× on the HybridQA grid, and 1.79× against 1.69× on CRAG, reaching at most 99% of the allowance at every level of either space and never passing it. The worst level of the fifteen-level sweep is in fact better on the larger space (1.53× to 1.45× on HybridQA, 2.11× to 2.07× on CRAG), because the extra plans give the action LP more interior points to mix. The gap to the offline optimum stays where it was while the optimum itself falls, which is the only monotonicity an operator can plan around: the larger space is never a liability, and whatever it offers is passed through.

7. Discussion

What the guarantee does and does not say Theorem 1 bounds the realised rate of the *declared* loss on the *served* stream. It does not say that the declared loss captures what the operator cares about; a contract on token-F1 is a contract on token-F1. It does not promise anything about a single answer, which is the nature of a rate. And it says nothing about the queries that were declined – if 40% of traffic is handed to a human, the guarantee covers the 60% that was served, and the decline rate is reported in every table precisely because it is the other half of the picture. An operator reading Table 1 should read cost, risk and decline rate together; a low cost bought with a 60% decline rate is a different service from a low cost bought with 5%.

Table 9
What enlarging the plan space does to a certificate, measured by the contracts it can actually honour rather than by its cost at levels chosen in advance. α is scanned on a 0.01 grid from 0.10 to 0.80 (71 levels) and each space is certified with the same randomised construction, calibration split and $\delta = 0.1$. *floor* is the risk of the best single plan, what the models and retrieval physically permit; α_{cert} is the strictest contract the certifier returns anything at, and the *slack* between them, in percentage points, is the range of contracts the space can meet but certification cannot reach. *levels* counts how many of the 71 the certifier serves. The ledger serves all 71 in every space, including every level below the floor, because declining carries no contract loss, so the column would read 71 four times and is omitted. Sampling that claim at 15 levels per space (Sec. 6.6) gives 60 cells, none breached, reaching 71–99.6% of the allowance. Enlarging the space moves these quantities in opposite directions on the two tasks – on HybridQA it costs one level and widens the gap, on CRAG it buys four levels and lets a contract 0.04 stricter be honoured – which is the point: the sign is a property of the task. Costs are mCNY per query at α_{cert}, where both mechanisms return something and the comparison is defined.

				what the certifier can honour			cost at α_{cert}	
Task	Space	$\lvert\mathcal{P}\rvert$	floor	α_{cert}	slack (pp)	levels	cert.	ledger
HybridQA	ladder	4	0.280	0.31	3.0	50	4.93	1.69
	grid	36	0.280	0.32	4.0	49	2.70	2.44
CRAG	ladder	4	0.579	0.63	5.1	18	1.60	0.40
	grid	24	0.536	0.59	5.4	22	1.45	0.42

The auditor is the real assumption Every accounting scheme is only as good as its ledger entries. Theorem 1 needs ℓ_t, and ℓ_t comes from an auditor – an LLM judge, an NLI model, a human. If the auditor is systematically biased, the ledger is systematically wrong and no amount of bookkeeping repairs it; this is a strictly stronger dependence than a calibration certificate has, since that one needs labels only during calibration. We regard this as the honest cost of a deterministic guarantee and have tried to price it rather than hide it: Section 6.5 measures what partial labelling costs under both accounting rules, and the audit line is inside every cost figure in the paper. The practical reading is that a deterministic contract is appropriate where auditing is already happening – regulated settings, services with human review queues – and that elsewhere the inverse-probability variant, with its probabilistic guarantee and its measured 6–59% breach rate, is the available compromise.

Why the ledger does not simply exhaust itself A natural worry is that the gate is a one-way ratchet: violations accumulate, the ledger empties, and the system declines forever. It does not, because declining earns allowance at rate α. If the deployed mixture has risk $r > \alpha$, the steady-state decline rate settles at $1 - \alpha/r$, which is exactly the minimum any feasible policy must have (Proposition 1); the controller discovers it without being told r. The measured decline rates track this prediction: on QAMPARI at $\alpha = 0.35$ with a risk floor of 0.591, the bound is 40.8% and the executor declines 51.9%, the gap being the cheaper, riskier plans it chooses to run in place of the safest one.

When this is the wrong design Three regimes favour the incumbent. If labels are unavailable after deployment at any price, the ledger cannot be maintained and a calibration certificate is the only option. If the stream is short – shorter than the few hundred queries the estimates need – the controller is still in its transient and a certificate calibrated offline may be cheaper; our ASQA results, on the shortest stream, show the smallest margin and the largest sensitivity to κ. And if declining is genuinely impossible, so that $\mathcal{A} = \mathcal{P}$, the gate has no safe action to fall back on and Theorem 1 does not apply; the best available then is the probabilistic guarantee, and the contract is capped at $\min_p r_p$ whatever one does.

Multi-dimensional contracts A service typically owes several rates at once – quality, citation support, freshness, latency. The construction extends directly: run one ledger per dimension and open the gate only when all of them can absorb a violation, which yields $V_t^{(j)} \le \alpha_j t$ simultaneously for every j with no union correction, since each bound is deterministic. The optimisation layer becomes an LP with d risk constraints, whose basic solutions have at most $d + 1$ nonzero weights. This is a strict simplification over the intersection–union testing a certified formulation needs, and it is the clearest illustration of what the reframing buys: the multiplicity problems that dominate the statistical design disappear when the guarantee stops being statistical. We report the single-dimension case here because our execution matrices carry per-query costs for one auditable loss per benchmark; the multi-ledger extension is implemented and left for evaluation on a workload with several audited dimensions.

Threats to validity Costs are list prices at a point in time and vendor pricing moves; we report the metering method and release the token counts so any table can be repriced. GPU-second pricing for local models depends on an assumed rental rate. The streams are permutations of finite pooled populations rather than fresh traffic, so the long-horizon results resample the same queries; this is why we report regret against horizon rather than claiming a steady state. Finally, the plan grids cover two of the four benchmarks, since executing 36 and 24 plans end to end on all four was beyond our budget; the four-benchmark results use the 69-candidate space over the ladder. Two grids are enough to show that the effect of enlarging the space has no

fixed sign, but not enough to predict its sign from properties of a task, which we regard as the more useful open question this raises.

8. Conclusion

A quality contract on a RAG service is an obligation about a realised rate, and this paper's claim is that such an obligation should be enforced rather than estimated. Admitting a decline action and gating execution on the unspent allowance $B_t = \alpha t - V_t$ makes $V_t \le \alpha t$ an identity of the trajectory rather than an event of probability $1 - \delta$: it holds under adversarial query orders, under mid-stream drift, and independently of how large the plan space is. What remains is a cost problem in which estimation errors are paid for in money, and we solve it with an optimistic linear program over plans plus declining, with $O(\sqrt{P \log T / T})$ regret.

The empirical consequences are consistent across four metered benchmarks and two plan grids built by unbundling retrieval from generation. The executor operates at 1.27–1.75× an offline optimum that knows every plan's risk, where one-shot certification with the same action space costs 2.89–5.85×; swept over 15 contract levels per task it honoured every one of them on every stream while spending 92–99% of the allowance, so the constraint binds without ever being crossed, where the certifier breached 545 of 1 200 streams and, once the traffic stopped being exchangeable with its calibration prefix, up to 100%. Enlarging the plan space sharpened the contrast in an unexpected way: the certified cost got worse on one grid and better on the other, so an operator cannot know in advance whether building a larger space will be rewarded, whereas the ledger's distance to the offline optimum stayed of the same order in both. The ablation is the part we would ask a sceptical reader to check first: giving the certifier the same decline action changes its cost by under 2%, and removing randomisation from the executor makes it worse than the certifier. None of the three ingredients works alone.

The design has a price and we have tried to state it plainly. A deterministic guarantee consumes the loss of every served query, so it presumes an auditor; auditing a quarter of traffic preserves the guarantee only if unlabelled queries are charged as failures, which costs declines, and the unbiased alternative breaches on 6–59% of streams. Where auditing already happens, this seems the right trade. Beyond RAG, the construction applies to any service with a rate obligation, a menu of differently priced execution strategies, and the right to decline – which is a fair description of most decision-support systems. We release the execution matrices, the token-exact cache and the code, so every number regenerates without further LLM calls.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

All code, execution matrices, calibration artifacts, and the token-exact LLM-call cache are publicly available at `https://github.com/xkk9866/contractRagDB`; every number in the paper regenerates from the artifact without further LLM calls.

CRediT authorship contribution statement

Kang Yao: Conceptualization, Methodology, Software, Formal analysis, Data curation, Visualization, Writing – original draft. **Shengjiang Zhang:** Methodology, Software, Validation, Investigation, Data curation, Writing – original draft, Writing – review & editing. **Ronghao Pei:** Software, Resources, Investigation, Validation, Writing – review & editing. **Weiwei Fu:** Supervision, Funding acquisition, Resources, Project administration, Writing – review & editing. **Jinjiang Cui:** Conceptualization, Supervision, Funding acquisition, Project administration, Writing – review & editing.

References

[1] A. N. Angelopoulos, S. Bates, E. J. Candès, M. I. Jordan, L. Lei, Learn then test: Calibrating predictive algorithms to achieve risk control, Annals of Applied Statistics (2025).

[2] S. Bates, A. N. Angelopoulos, L. Lei, J. Malik, M. I. Jordan, Distribution-free, risk-controlling prediction sets, Journal of the ACM 68 (2021) 1–34.

[3] S. Li, S. Park, I. Lee, O. Bastani, TRAQ: Trustworthy retrieval augmented question answering via conformal prediction, in: NAACL, 2024.

[4] M. Kang, N. M. Gürel, N. Yu, D. Song, B. Li, C-RAG: Certified generation risks for retrieval-augmented language models, in: ICML, 2024.

[5] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, T. Neumann, How good are query optimizers, really?, Proceedings of the VLDB Endowment 9 (2015) 204–215.

[6] V. Vovk, A. Gammerman, G. Shafer, Algorithmic Learning in a Random World, Springer, 2005.

[7] A. N. Angelopoulos, S. Bates, A gentle introduction to conformal prediction and distribution-free uncertainty quantification, Foundations and Trends in Machine Learning 16 (2023) 494–591.

[8] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei, T. Schuster, Conformal risk control, in: International Conference on Learning Representations (ICLR), 2024.

[9] C. Mohri, T. Hashimoto, Language models with conformal factuality guarantees, in: International Conference on Machine Learning (ICML), 2024.

[10] J. J. Cherian, I. Gibbs, E. J. Candès, Large language model validity via enhanced conformal prediction methods, in: Advances in Neural Information Processing Systems (NeurIPS), 2024.

[11] R. F. Barber, E. J. Candès, A. Ramdas, R. J. Tibshirani, Conformal prediction beyond exchangeability, The Annals of Statistics 51 (2023) 816–845.

[12] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, nonparametric, nonasymptotic confidence sequences, The Annals of Statistics 49 (2021) 1055–1080.

[13] A. Ramdas, P. Grünwald, V. Vovk, G. Shafer, Game-theoretic statistics and safe anytime-valid inference, Statistical Science 38 (2023) 576–601.

[14] I. Waudby-Smith, A. Ramdas, Estimating means of bounded random variables by betting, JRSS-B 86 (2024).

[15] J. Shin, A. Ramdas, A. Rinaldo, E-detectors: A nonparametric framework for sequential change detection, The New England Journal of Statistics in Data Science 2 (2024).
[16] H. Khosravi, X. Huo, Conformal selective acting: Anytime-valid risk control for RLVR-trained LLMs, arXiv preprint arXiv:2605.20270 (2026).
[17] C. K. Chow, On optimum recognition error and reject tradeoff, IEEE Transactions on Information Theory 16 (1970) 41–46.
[18] Y. Geifman, R. El-Yaniv, Selective classification for deep neural networks, in: Advances in Neural Information Processing Systems (NeurIPS), 2017.
[19] R. El-Yaniv, Y. Wiener, On the foundations of noise-free selective classification, Journal of Machine Learning Research 11 (2010) 1605–1641.
[20] A. Badanidiyuru, R. Kleinberg, A. Slivkins, Bandits with knapsacks, Journal of the ACM 65 (2018) 1–55.
[21] S. Agrawal, N. R. Devanur, Bandits with concave rewards and convex knapsacks, in: Proceedings of the ACM Conference on Economics and Computation (EC), 2014, pp. 989–1006.
[22] N. R. Devanur, K. Jain, B. Sivan, C. A. Wilkens, Near optimal online algorithms and fast approximation algorithms for resource allocation problems, Journal of the ACM 66 (2019) 1–41.
[23] L. Chen, M. Zaharia, J. Zou, FrugalGPT: How to use large language models while reducing cost and improving performance, arXiv:2305.05176 (2023).
[24] D. Ding, A. Mallick, C. Wang, R. Sim, S. Mukherjee, V. Rühle, L. V. S. Lakshmanan, A. H. Awadallah, Hybrid LLM: Cost-efficient and quality-aware query routing, in: International Conference on Learning Representations (ICLR), 2024.
[25] Q. J. Hu, J. Bieker, X. Li, N. Jiang, B. Keigwin, G. Ranganath, K. Keutzer, S. K. Upadhyay, Routerbench: A benchmark for multi-LLM routing system, arXiv preprint arXiv:2403.12031 (2024).
[26] S. Jeong, J. Baek, S. Cho, S. J. Hwang, J. C. Park, Adaptive-RAG: Learning to adapt retrieval-augmented large language models through question complexity, in: NAACL, 2024.
[27] M. Russo, S. Sudhir, G. Vitagliano, C. Liu, T. Kraska, S. Madden, M. Cafarella, Abacus: A cost-based optimizer for semantic operator systems, PVLDB 18 (2025).
[28] C. Jin, Z. Zhang, X. Jiang, F. Liu, X. Liu, X. Liu, X. Jin, RAGCache: Efficient knowledge caching for retrieval-augmented generation, arXiv preprint arXiv:2404.12457 (2024).
[29] R. Avnur, J. M. Hellerstein, Eddies: Continuously adaptive query processing, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2000, pp. 261–272.
[30] S. Babu, P. Bizarro, D. DeWitt, Proactive re-optimization, in: Proceedings of the ACM SIGMOD International Conference on Management of Data, 2005, pp. 107–118.
[31] V. Markl, V. Raman, D. Simmen, G. Lohman, H. Pirahesh, M. Cilimdzic, Robust query processing through progressive optimization, in: SIGMOD, 2004.
[32] W. Chen, H. Zha, Z. Chen, W. Xiong, H. Wang, W. Y. Wang, HybridQA: A dataset of multi-hop question answering over tabular and textual data, in: Findings of EMNLP, 2020.
[33] X. Yang, K. Sun, H. Xin, Y. Sun, N. Bhalla, X. Chen, S. Choudhary, R. D. Gui, Z. W. Jiang, Z. Jiang, et al., CRAG – comprehensive RAG benchmark, in: NeurIPS Datasets and Benchmarks, 2024.
[34] I. Stelmakh, Y. Luan, B. Dhingra, M.-W. Chang, ASQA: Factoid questions meet long-form answers, in: EMNLP, 2022.
[35] T. Gao, H. Yen, J. Yu, D. Chen, Enabling large language models to generate text with citations, in: EMNLP, 2023.
[36] O. Honovich, R. Aharoni, J. Herzig, H. Taitelbaum, D. Kukliansy, V. Cohen, T. Scialom, I. Szpektor, A. Hassidim, Y. Matias, TRUE: Re-evaluating factual consistency evaluation, in: NAACL, 2022.
[37] S. J. Amouyal, T. Wolfson, O. Rubin, O. Yoran, J. Herzig, J. Berant, QAMPARI: A benchmark for open-domain questions with many answers from multiple paragraphs, in: Proceedings of the Third Workshop on Natural Language Generation, Evaluation, and Metrics (GEM), 2023.
[38] M. Raasveldt, H. Mühleisen, DuckDB: An embeddable analytical database, in: SIGMOD, 2019.
[39] S. Robertson, H. Zaragoza, The probabilistic relevance framework: BM25 and beyond, Foundations and Trends in Information Retrieval 3 (2009).
[40] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, C-Pack: Packaged resources to advance general Chinese embedding, arXiv:2309.07597 (2023).
[41] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (2019).
[42] G. V. Cormack, C. L. A. Clarke, S. Buettcher, Reciprocal rank fusion outperforms Condorcet and individual rank learning methods, in: SIGIR, 2009.